FocusLock: Engineering Guidebook

Webcam-Based Gaze Control System — Complete 6-Stage Architecture & Viva Reference

1. Executive Overview & Core Philosophy

FocusLock is an educational computer-vision project designed to achieve hands-free computer interaction using standard webcam hardware. Unlike commercial eye-trackers (which rely on infrared illumination and specialized cameras), FocusLock uses standard RGB imagery processed through a structured 6-layer engineering pipeline.

> ***Crucial College Demo Concept:*** *MediaPipe is only the sensor. The true novelty and engineering value of this project lies in the downstream stages:* ***Coordinate Normalization***, ***Temporal Filtering (Jitter Reduction)***, ***5-Point Linear Calibration Mapping***, ***Dwell-Based Intent Detection***, ***OS Actuation Integration***, *and* ***Empirical Telemetry Benchmarking***.

2. Full Architectural Pipeline

Layer	Module	Primary Function	Key Math / Technique
1. Sensing	iris_tracker.py	Extract 478 face & iris landmarks	MediaPipe Tasks FaceLandmarker (RGB)
2. Normalization	gaze_estimator.py	Make iris position relative to eye socket	Ratio: (iris - min) / (max - min)
3. Filtering	moving_average.py	Eliminate sensor noise & micro-saccades	Sliding Window FIFO Mean (N=7)
4. Calibration	calibrator.py	Map normalized gaze to screen coordinates	1D Linear Least Squares (numpy.polyfit)
5. Intent	dwell_detector.py	Prevent unintended clicks (Midas Touch)	Continuous Zone Dwell Timer (1.8s)
6. Actuation	presentation_controller.py	Simulate presentation keypresses	PyAutoGUI Left/Right Arrow Dispatch
7. Telemetry	logger.py / plot_benchmarks.py	Frame logging & scientific graphing	Pandas CSV logging & Matplotlib plots

3. Stage 1: Sensing & Iris Extraction

- **Modern MediaPipe Tasks API:** Uses FaceLandmarker with standalone face_landmarker.task model bundle (~30MB), providing 478 3-D landmarks.
- **Iris Center Indices:** Landmark **468** represents the right iris center and landmark **473** represents the left iris center (in anatomical orientation).
- **BGR to RGB:** OpenCV decodes in BGR; MediaPipe requires RGB (cv2.cvtColor is mandatory).

4. Stage 2: Gaze Normalization & Signal Smoothing

- **Spatial Normalization:** Gaze_X = (Iris_X - X_min) / (X_max - X_min) maps iris inside [0.0, 1.0]. Because both the eyelid bbox and iris shift during head movement, the ratio remains partially head-pose invariant.
- **Dual-Eye Fusion:** Averages left and right eye vectors to mitigate unilateral eyelid occlusion.
- **Moving Average Filter:** FIFO sliding-window (N=7 frames) cancels high-frequency sensor noise and microsaccadic tremor with ~230ms perceptual latency.

5. Stage 3: 5-Point Calibration & Screen Mapping

- **Why Required:** Physiological iris rotation only spans a narrow band (0.35 to 0.65) in normalized space. Calibration calculates user-specific Gain (m) and Offset (c).
- **Linear Regression:** X_screen = m_x * Gaze_X + c_x via numpy.polyfit across 5 target targets, saved in calibration.json.

6. Stage 4: Dwell-Based Intent Detection

- **The Midas Touch Problem:** Human eyes are perceptual organs. Looking at the screen edge must not fire instant commands. We enforce a continuous 1.8s dwell threshold.
- **Single-Shot Trigger Protection:** Once 1.8s elapses, the action fires exactly once and locks further triggers until the user looks away from the zone.

7. Stage 5: Presentation Control & DEMO_MODE Safety

- **PyAutoGUI Actuation:** Maps NEXT_SLIDE $ o$ Right Arrow, and PREV_SLIDE $ o$ Left Arrow.
- **DEMO_MODE Safety Toggle ('d' key):** Allows testing without sending active keystrokes to other open computer windows.

8. Stage 6: Telemetry Logging & Benchmark Graphing

- **Frame Logging:** Writes timestamp, raw, filtered, calibrated coords, zone, and actions to data/logs/gaze_log.csv.
- **Matplotlib Benchmark:** plot_benchmarks.py generates 3-panel scientific figures (Raw vs. Filtered jitter reduction, Calibrated timeline with action markers, and 2D spatial trajectory).

9. Viva & Demonstration Q&A; Cheatsheet

Question from Evaluator	Strong Engineering Response
Why did you not use a Deep Learning classifier for gaze?	Custom neural networks require thousands of labeled user gaze images and high GPU compute. Our geometric pipeline (MediaPipe landmarks + relative normalization + linear calibration) is lightweight, runs in real-time on CPU at 30+ FPS, and provides explicit mathematical interpretability.
What is the 'Midas Touch' problem in gaze interaction?	The human eye is primarily a sensory organ, not an actuator. Everywhere the user looks cannot trigger a click. Dwell-time intent detection ensures only deliberate, sustained focus triggers a system command.
What are the real-world limitations of this system?	1. Variable room illumination alters eyelid contours. 2. Severe head rotations cause calibration drift. 3. Eyeglasses reflections can briefly occlude iris landmarks. 4. First-order linear mapping assumes flat planar projection rather than spherical eye geometry.